01 · ACCESS

Getting started on the Illinois Campus Cluster

A practical workflow for data analysis



Illinois Computes · Official photograph

One small analysis from input to result

Input: data/samples.csv

```
sample_id,reads
sample_A,1000
sample_B,1500
sample_C,1200
```

Expected result: results/summary.csv

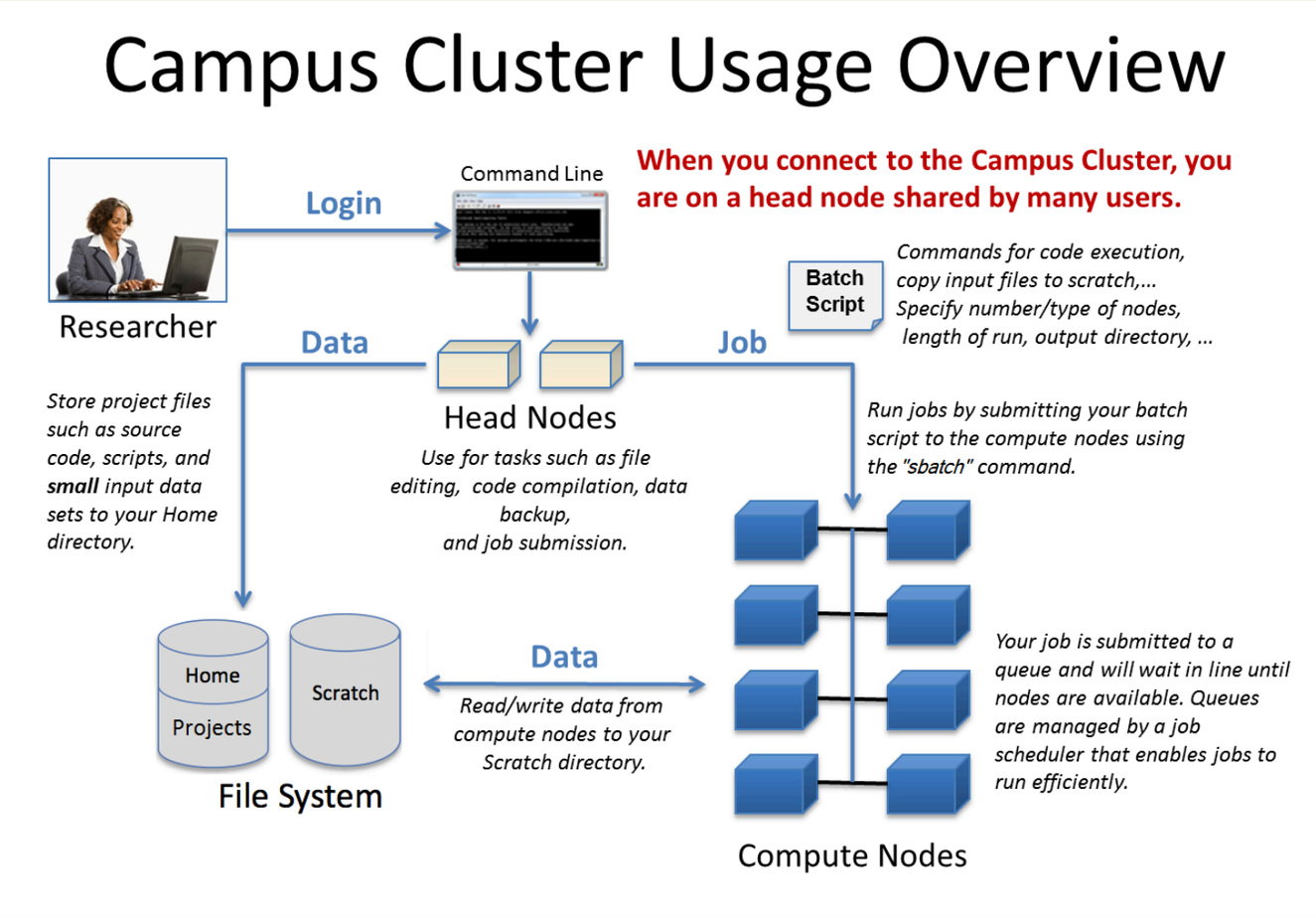
```
samples,total_reads
3,3700
```

The program analysis.py reads the table and adds the reads.

The job file analysis.sbatch asks Slurm, the job scheduler, to run it.

> We will move this project onto the cluster and produce this result.

Your laptop, the login node, and the compute node



HPC = high-performance computing. A node is one computer. Login: prepare files. Compute: run scheduled analysis.

Account access and the details you need

Existing lab access

Ask the PI or technical contact which investor group to join.

A new computing need

Start an Illinois Computes request.

Keep this detail	Used for
NetID	Your login identity
Slurm account	The group’s compute allocation
Partition	A queue you may submit to
Project path	An approved writable folder

Research access request ↗ Illinois Computes request ↗

Wait for approval and confirm these details and Duo access before connecting.

The first SSH login

On your laptop: open Terminal (macOS) or PowerShell (Windows)

```
ssh YOUR_NETID@cc-login.campuscluster.illinois.edu
```

`ssh` starts the connection · before `@`: your user name · after `@`: the cluster address

1 · Run the command

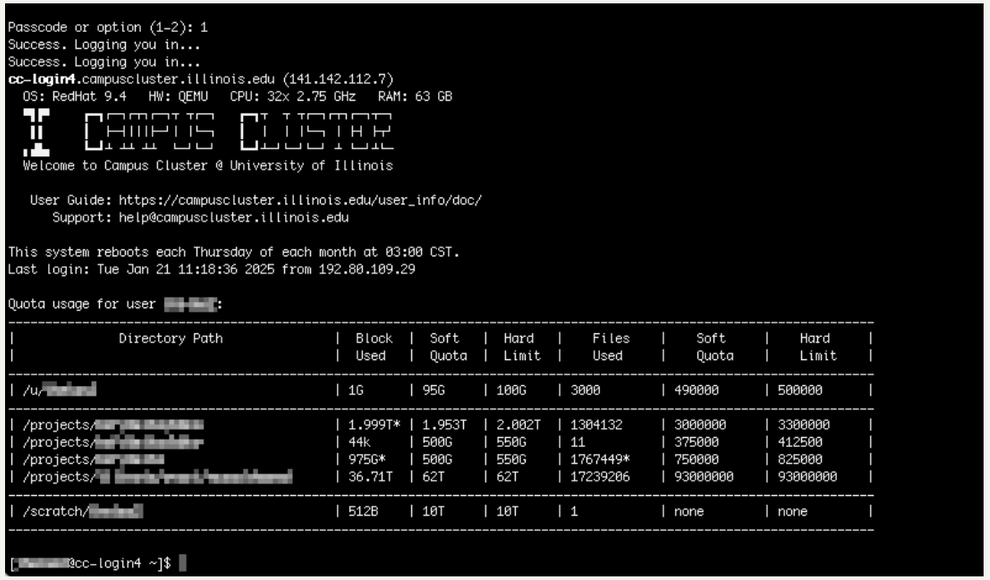
Replace YOUR_NETID, then press Enter.

2 · Password

Type your NetID password and press Enter; typing stays hidden.

3 · Duo

Follow the prompt and approve the authentication request.



A welcome message and remote prompt indicate that login succeeded. Banner values may be historical.

Local and remote: where things happen

LOCAL

Your laptop

VS Code window

View and edit

`analysis.py`

Your screen, keyboard, and mouse

Remote - SSH



View · edit · save

REMOTE

The cluster

Project files

`demo/analysis.py`

Save updates this file.

Remote terminal

Commands run on the cluster.

In the SSH window, you work with files and a terminal on the cluster.

Prepare VS Code on your laptop

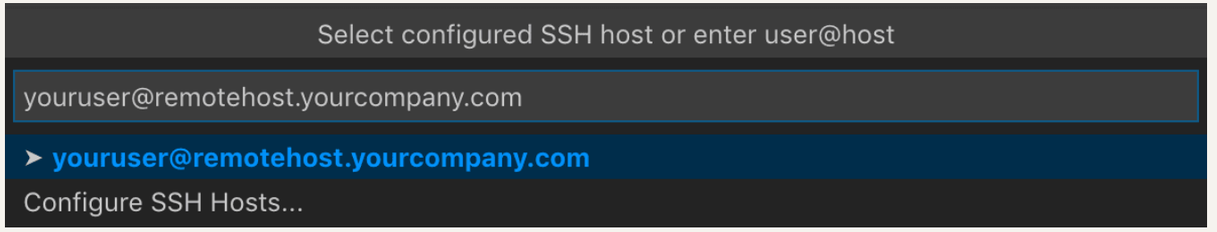
Install VS Code	Download the desktop application for your operating system.
Open Extensions	Click the Extensions icon in VS Code’s left sidebar.
Search	Enter Remote - SSH; choose the extension published by Microsoft.
Install	Click Install and wait until it finishes.

Official VS Code setup ↗

> This installs the editor and connection tool on your laptop.

Connecting VS Code with Remote - SSH

Command Palette	Cmd+Shift+P on macOS; Ctrl+Shift+P on Windows/Linux
Choose	Remote-SSH: Connect to Host...
Enter and sign in	YOUR_NETID@cc-login.campuscluster.illinois.edu



Microsoft example UI. Enter your NetID and the ICC hostname in this field.

Complete password + Duo. Choose Linux if asked. The lower-left indicator should show SSH and the host.

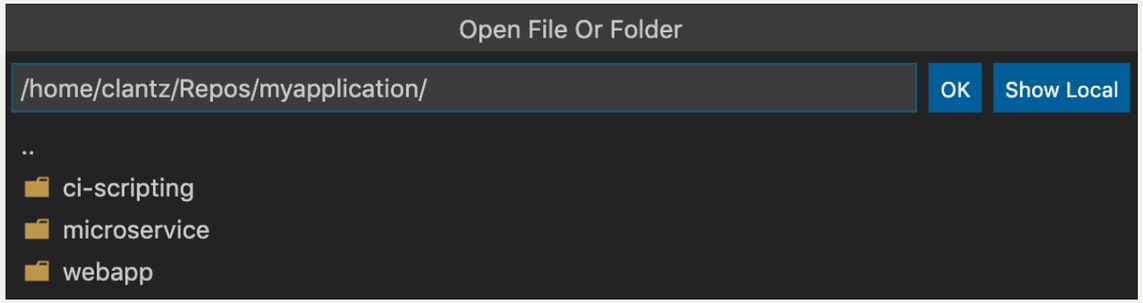
Choose where the project will live

Location	Use it for	Remember
/u/YOUR_NETID	Settings and small personal files	Home has a quota.
/projects/YOUR_GROUP	Group data, software, and retained outputs	Use your assigned writable location.
/scratch/YOUR_NETID	Temporary working data	Files can be purged.

> For this example: `/projects/YOUR_GROUP/demo`

Open the remote folder and upload the project

On your laptop: download the course files ↗, unzip them, and find `HPC_Introduction/demo`.



Example UI: use your assigned project path.

1 · Remote destination

File → Open Folder → /projects/YOUR_GROUP

2 · Transfer

Drag the local demo folder into remote Explorer; wait until finished.

3 · Open the project

File → Open Folder → /projects/YOUR_GROUP/demo

4 · Check

Open data/samples.csv: the header and three sample rows should appear.

> The SSH indicator identifies the cluster. Cmd+S / Ctrl+S saves edits directly to the remote file.

The files now in the remote demo folder

Uploaded files

```
demo/
  data/samples.csv
  analysis.py
  analysis.sbatch
  hello.sbatch
  notes.txt
  README.md
```

data/samples.csv

The same three-row input table

analysis.py

The calculation we want to run

hello.sbatch

A first job that checks submission

analysis.sbatch

The job that will run the calculation

> The results folder and Python environment will be created in later steps.

Open a terminal in the same project

VS Code → Terminal → New Terminal

```
hostname
cd /projects/YOUR_GROUP/demo
pwd
```

hostname	Check which computer is running the shell.
cd	Use the same remote demo folder as Explorer.
pwd	Confirm the current folder before running commands.

> Run one line, press Enter, wait for the prompt, then continue.

Tab: complete a path · ↑: recall a command · **Ctrl+C**: stop the foreground command

Read a command and inspect the input

```
head -n 5 data/samples.csv
```

Program

head shows the beginning of a text file.

Option + value

-n 5 requests the first five lines.

File path

data/samples.csv selects the input.

Output: this file has only four lines

```
sample_id,reads
sample_A,1000
sample_B,1500
sample_C,1200
```

`cat notes.txt` : print a small file · `less notes.txt` : browse text; press `q` to leave

Quote paths containing spaces. Use `head --help` or `man head` for options (`q` closes the manual).

Navigate the uploaded project

Transcript: type commands after $; other lines are output

```
$ pwd
/projects/YOUR_GROUP/demo
$ cd data
$ pwd
/projects/YOUR_GROUP/demo/data
$ cd ..
$ pwd
/projects/YOUR_GROUP/demo
$ cd ~
$ pwd
/u/YOUR_NETID
$ cd -
/projects/YOUR_GROUP/demo
$ pwd
/projects/YOUR_GROUP/demo
```

Command	Meaning
pwd	Print the current folder
cd data	Enter the data subfolder
cd ..	Go up one folder
cd ~	Go to your home folder
cd -	Return to the previous folder

`cd -` also prints its destination. The next `pwd` confirms it.

`.` current folder · `..` parent · `/projects/…` absolute path · `data/…` relative path

List files and inspect their details

Command	What changes
ls	List visible names
ls -a	Include dot-prefixed entries
ls -l	Show details such as size and date
ls -lh	Use readable sizes, such as K or M
ls -lah	Combine all three options

Illustrative listing lines; total lines omitted

```
$ ls data
samples.csv
$ ls -a data
.  ..  samples.csv
$ ls -l data
-rw-r--r-- 1 user group 58 Oct 27 10:00 samples.csv
$ ls -lh data
-rw-r--r-- 1 user group 58 Oct 27 10:00 samples.csv
$ ls -lah data
drwxr-xr-x 2 user group 4.0K Oct 27 10:00 .
drwxr-xr-x 3 user group 4.0K Oct 27 10:00 ..
-rw-r--r-- 1 user group   58 Oct 27 10:00 samples.csv
```

The CSV is only **58 bytes**, so `-h` still shows 58. Later, `ls -a` will reveal the `.venv` folder we create.

Creating, copying, moving, and deleting

Command	Result
`mkdir results`	Create one new folder
`mkdir -p results/qc`	Create missing parent folders; allow existing folders
`cp -i notes.txt notes_backup.txt`	Copy a file; ask before overwriting
`cp -r data data_backup`	Copy a folder and its contents to a new folder
`mv -i notes_backup.txt notes_old.txt`	Rename or move; ask before overwriting
`rm -i notes_old.txt`	Ask before deleting this file

`cp` and `mv`: source first, destination second. Shell deletion normally has no Trash recovery.

Find a file, then find text inside it

Search filenames within data and its subfolders

```
find data -type f -name "*.csv"
```

-type f	Match files; -name tests their names against the quoted pattern.
"*.csv"	The * matches any text before .csv.

Search inside this file; -n includes line numbers

```
grep -n "sample_B" data/samples.csv
```

Expected grep output: line 3 contains the match

```
3:sample_B,1500
```

Count lines and check disk usage

Run this command

```
wc -l data/samples.csv
```

wc -l

Count lines in this text file.

Expected output; do not type this line

```
4 data/samples.csv
```

4 lines

One header plus three sample rows.

Check space used by the input folder

```
du -sh data
```

`-s` gives one summary total · `-h` uses readable units such as K or M

> Line count is not automatically sample count. Disk usage is not your remaining quota.

Slurm runs work on allocated resources

LOGIN NODE

Prepare and submit

`hello.sbatch`

Your terminal stays here.

`sbatch` →

Slurm

Wait for resources

The scheduler

Allocate →

COMPUTE NODE

Run the job

`hello.sbatch`

The program runs here.

↕ ↕

Same shared `demo/` folder Input · scripts · output

Batch: submit a script. **Interactive:** request a compute shell with `srun`.

> Submit on the login node; the job runs on an allocated compute node.

A small batch script: hello.sbatch

```
01 #!/bin/bash
02 #SBATCH --job-name=hello
03 #SBATCH --account=YOUR_ACCOUNT
04 #SBATCH --partition=YOUR_PARTITION
05 #SBATCH --nodes=1
06 #SBATCH --ntasks=1
07 #SBATCH --cpus-per-task=1
08 #SBATCH --mem=1G
09 #SBATCH --time=00:05:00
10 echo "My first HPC job"
```

01 Use Bash to read the commands.

02 Give the job a recognizable name.

03 Choose your assigned allocation.

04 Choose an eligible queue.

05 Request one compute node.

06 Request one task for this program.

07 Reserve one CPU for that task.

08 Request 1 GiB of RAM per node.

09 Set a five-minute running-time limit.

10 Print a message into the job output.

Edit these lines inside hello.sbatch in VS Code, then save. Submit it from the terminal after reviewing the resource settings.

Nodes, tasks, CPUs, memory, and time

Example resource request: count the node, the tasks, then the CPUs.

`--nodes=1`

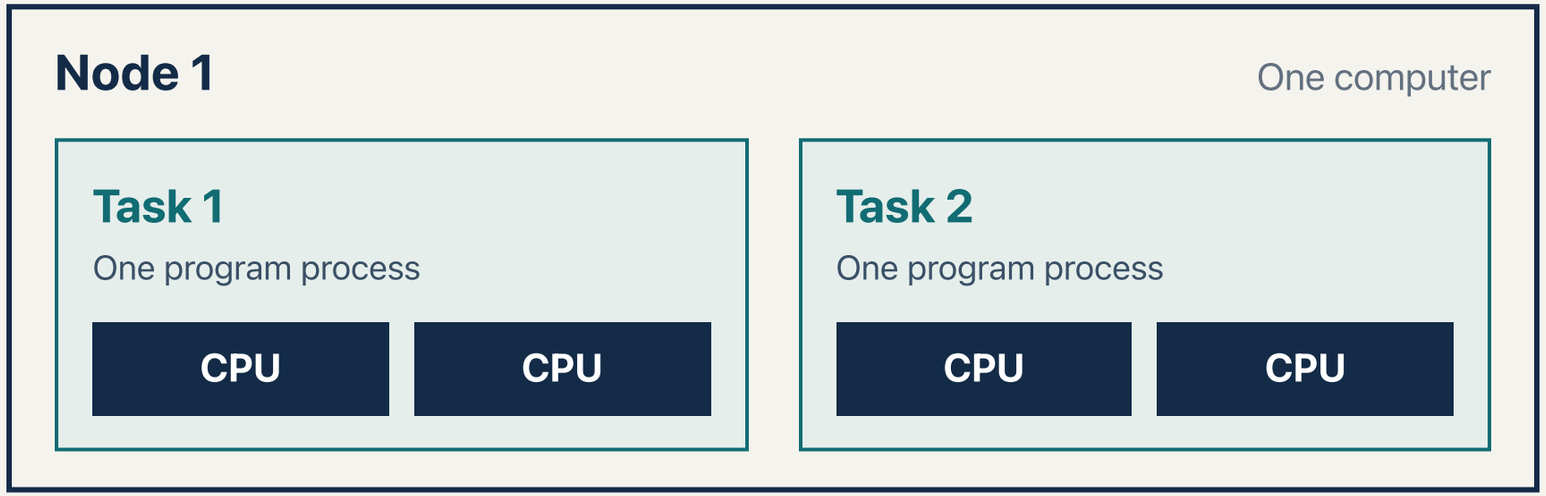
1 computer

`--ntasks=2`

2 tasks in the whole job

`--cpus-per-task=2`

2 CPUs reserved for each task



> **2 tasks × 2 CPUs/task = 4 CPUs requested**
>
> ntasks is already the total; do not multiply by nodes again.

Reserving resources does not launch the tasks or make the program parallel.

Keep hello.sbatch at: 1 node · 1 task · 1 CPU/task

`--mem=1G` 1 GiB of RAM per node

`--time=00:05:00` Runtime limit: hours:minutes:seconds

Submit hello.sbatch and check the queue

After replacing account/partition and saving hello.sbatch

```
$ pwd
/projects/YOUR_GROUP/demo
$ sbatch hello.sbatch
Submitted batch job 123456
```

-u selects your jobs; run again to refresh

```
squeue -u YOUR_NETID
```

State	Meaning
PD	Pending: waiting for resources or another scheduling condition
R	Running on allocated resources

`scancel 123456` stops an unwanted job. Let this hello job finish; use its actual returned ID.

> When the job leaves the queue, check its final record and output on the next slide.

Check the first job's message

-j selects the job; --format chooses the displayed columns

```
sacct -j 123456 --format=JobID,State,ExitCode
```

Illustrative successful record

```
JobID    State        ExitCode
123456   COMPLETED    0:0
```

`0:0` means exit status 0 and no terminating signal. Use your actual job ID.

Expected text: My first HPC job

```
cat slurm-123456.out
```

> Continue when the job has completed and the expected message is present.

Get a compute shell for software setup

From the login shell in demo: paste this whole block as one command

```
srun --account=YOUR_ACCOUNT --partition=YOUR_PARTITION \
  --nodes=1 --ntasks=1 --cpus-per-task=2 \
  --mem=4G --time=00:15:00 --pty bash
```

srun	Start a program on allocated compute resources.
--pty bash	Open an interactive terminal running the Bash shell.
After it starts	Run hostname and pwd. Keep this shell open for Python setup.

A trailing `\` continues a command onto the next line; nothing follows it on that line.

Make Python available in the compute shell

Inside the allocation, in demo; use a version shown as available

```
module avail python
module load python/3.11.11
module list
```

Command	Meaning
module avail python	List cluster modules matching Python
module load python/3.11.11	Make this Python version available in this shell
module list	Confirm which cluster modules are loaded

> analysis.py imports pandas. Next, create a place to install that package.

Create .venv and install pandas

Continue in the same compute shell and remote demo folder

```
python -m venv .venv
source .venv/bin/activate
python -m pip install pandas
ls -a
```

Create .venv	-m runs Python’s venv utility; .venv is the new folder.
Activate	source applies the activation script to this current shell.
Install pandas	-m pip runs pip with this Python; install names the action.

Wait for pip to finish and the prompt to return. Then check the import on the next slide.

Verify pandas and record its version

-c executes the quoted Python code immediately

```
python -c "import pandas; print(pandas.__version__)"
```

import pandas	Check that this Python can load the package.
Print version	print(pandas.__version__) displays the installed version; expect a version number.

freeze lists installed package versions; > writes them into this file

```
python -m pip freeze > requirements.txt
```

> Continue only after the import succeeds. Keep this shell open for the next slide.

The environment stays; activation is per shell

Install packages

```
python -m pip install pandas
```

After creating and activating .venv

Write files →

Shared project storage

`demo/.venv/`

Python + installed packages

These files remain.

Activate in each new shell or job

`source .venv/bin/activate`

Current compute shell

`python` → `.venv/bin/python`

Same existing

.venv

No reinstall

A later batch job

`python` → `.venv/bin/python`

> Closing a shell keeps .venv. A new shell or job selects the environment again.

Return to the login shell

Exit the allocated shell once, then check where you are

```
exit
hostname
pwd
```

hostname	Should identify the original login host again.
pwd	Should still identify /projects/YOUR_GROUP/demo.
If the folder differs	cd back to your approved demo path.

> The Python project is ready. Next prepare analysis.sbatch in this login shell.

Prepare analysis.sbatch for the same project

1 · Beginning of analysis.sbatch

```
#!/bin/bash
#SBATCH --job-name=analysis
#SBATCH --account=YOUR_ACCOUNT
#SBATCH --partition=YOUR_PARTITION
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:05:00
```

2 · Continue in the same file, after the left column

```
#SBATCH --output=analysis-%j.out
#SBATCH --error=analysis-%j.err

set -e
module load python/3.11.11
source .venv/bin/activate
python analysis.py
```

Use the **same Python module version** as during installation. `set -e` stops this script if setup fails.

> Replace account and partition, save the file, then submit from the login shell in demo.

Submit the analysis job

In the login shell, from the demo folder

```
pwd
ls data/samples.csv analysis.py analysis.sbatch
sbatch analysis.sbatch
```

Illustrative analysis job ID; hello used 123456

```
Submitted batch job 123457
```

Check whether this new job is pending or running

```
squeue -u YOUR_NETID
```

> Use this new job ID for the analysis logs and completion checks.

Follow the analysis log

After the analysis job starts and the log exists

```
tail -n 20 analysis-123457.out
tail -f analysis-123457.out
```

First press Ctrl+C to stop tail; then search the error log

```
grep -in "error" analysis-123457.err
```

Ctrl+C	Stop tail; the submitted job keeps running.
Need to cancel?	scancel 123457 cancels this analysis job.

> tail: -n 20 = last 20 lines, -f = follow new output · grep: -i = ignore case, -n = line numbers

Check the state and the result file

After the analysis job finishes

```
sacct -j 123457 --format=JobID,State,ExitCode
cat results/summary.csv
```

Expected file contents

```
samples,total_reads
3,3700
```

Job record

COMPLETED and ExitCode 0:0

Output log

Saved results/summary.csv

Input comparison

3 rows; 1000 + 1500 + 1200 = 3700

> The result should match the goal shown at the start of the lecture.

Open the result in VS Code

Key files after the successful analysis

```
demo/
  data/samples.csv
  analysis.py
  analysis.sbatch
  .venv/
  requirements.txt
  analysis-123457.out
  analysis-123457.err
  results/summary.csv
```

Explorer → results

Open summary.csv: 3 samples and 3700 total_reads.

Keep it remote

Read or edit without downloading first.

Need a local copy?

Right-click summary.csv → Download...

Logs and images open directly. Use viewer extensions for PDF, CSV or notebooks.

> The Python workflow is complete: input, environment, submitted job, logs, and checked output.

Microsoft · Remote - SSH · Rainbow CSV · Extension documentation · vscode-pdf · Extension documentation · Microsoft · Jupyter extension · Microsoft · File import and download implementation

Optional: GPU and named-node requests

A job that needs a GPU

```
#SBATCH --partition=YOUR_GPU_PARTITION
#SBATCH --gres=gpu:GPU_TYPE:1
```

`--gres` : generic resource · `gpu:TYPE:1` : one GPU of that type per node

Inside the allocated NVIDIA GPU job: `nvidia-smi` shows the visible device.

A job that needs a particular node

```
#SBATCH --nodes=1
#SBATCH --nodelist=NODE_NAME
```

`--nodes` : how many computers · `--nodelist` : which named computer

`sinfo -p YOUR_PARTITION -N` lists nodes in that partition.

> Use these only when the project requires them. The completed Python demo needs neither.

For an R project: install and load a package

1 · In a separate allocated shell, start R

```
module load R
export R_LIBS_USER="$PWD/R-library"
mkdir -p "$R_LIBS_USER"
R
```

2 · In the R console, install, check, and quit

```
install.packages("ggplot2",
  repos = "https://cloud.r-project.org")
library(ggplot2)
q(save = "no")
```

`install.packages()` installs package files. `library()` loads the package in each R session.

`R_LIBS_USER`: project package folder · `repos`: download site

> 3 · After R closes, type `exit` at the shell prompt to release the allocation.

For an R project: change the job body

R commands below the #SBATCH resource settings

```
module load R
export R_LIBS_USER="$PWD/R-library"
Rscript analysis.R
```

After copying and configuring the R templates from examples/

```
sbatch analysis_R.sbatch
```

Same input	data/samples.csv
R output	results/summary_R.csv: 3 samples and 3700 total_reads

> Use the same R version and library location used during package setup.

Common errors and the first useful check

What you see	Check first
No such file or directory	pwd, the path, and the filename’s capitalization
Command not found	The required module or environment
No module named… / no package called…	The active Python or R library, then installation
OUT_OF_MEMORY / TIMEOUT	Requested RAM or runtime and the tool’s actual needs
Invalid account / partition	The allocation and queue pair assigned to you
Permission denied / quota exceeded	The target folder’s access and available quota

Documentation and help

ICC · Accessing the System

ICC · Running Jobs

ICC · Software and Modules

Missing Semester · Introduction to the Shell (2026)

Microsoft · Remote - SSH

ICC support: **help@campuscluster.illinois.edu**

Detailed references: SOURCES.md · Optional templates and Conda setup: examples/README.md